

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a6ca639f8bf684b2d.jsonl`
- SHA-256: `43bfe7c735474b99d7f7ee150792fa16356d56e083dd5ccd7f146a6da906efad`
- Source modified: `2026-06-22T09:50:09+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:32.825Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "backend.storage.fetch() signature and gs:// handling verified correct",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/storage/__init__.py:106-111, nightly_reelcount.py:277-278",
  "severity": "low",
  "detail": "fetch(uri, dst=None, config=None, overwrite=False) is called at line 277-278 with video_uri. For gs:// URIs, the fetch() dispatches to GCSStorage.fetch_to_local() (gcs.py:30-39), which downloads to dst if provided. The nightly runner passes only uri + labels_ref (for _load_gts), both handled correctly by the URI-driven backend.",
  "fix": "No fix needed. Storage integration is correct."
}
```

2. assistant (2026-06-22T09:48:34.969Z)

I'll examine the NIGHTLY REEL-COUNT engine-regression change, focusing on the alleged finding about `backend.storage.fetch()` signature and gs:// handling. Let me read the relevant files.

3. user (2026-06-22T09:48:36.226Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
2      nightly).
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
4      deployment serves
5      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
6      not change**
7      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
8      of CACHED
9      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
```

```

model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10    What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11    ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see Sguard).
14    ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16    ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18    Extensible pattern (add a video = one manifest row + re-freeze):
19    ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20    ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22    Usage
23    -----
24    Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25    python scripts/nightly_reelcount.py --update-baseline
26
27    Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28    python scripts/nightly_reelcount.py --email
29
30    Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31    (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32    """
33    from __future__ import annotations
34
35    import argparse
36    import datetime as _dt
37    import json
38    import os
39    import sys
40    from dataclasses import dataclass, field
41    from typing import Any, Dict, List, Optional, Tuple
42
43    REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44    if REPO_ROOT not in sys.path:
45        sys.path.insert(0, REPO_ROOT)
46
47    # billing is pure (typing only) ? safe to import at module top.
48    from backend import billing # noqa: E402
49
50    DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51    DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52    DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54    #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.

```

```

55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111                 "n": len(per_video)}
112

```

```

113
114 @dataclass
115 class Finding:
116     video: str
117     metric: str          # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118     kind: str           # "regression" | "improvement" | "stale"
119     baseline: Optional[float] = None
120     current: Optional[float] = None
121     detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134 @dataclass
135 class NightlyResult:
136     findings: List[Finding] = field(default_factory=list)
137     added: List[str] = field(default_factory=list)
138     removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).

```

```

174     """
175     res = NightlyResult()
176     if baseline.get("segmenter") != current.get("segmenter"):
177         res.findings.append(Finding("", "segmenter", "stale",
178                                   detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}}"))
179     return res
180
181     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182     res.added = sorted(set(c_pv) - set(b_pv))
183     res.removed = sorted(set(b_pv) - set(c_pv))
184     if res.added or res.removed:
185         res.findings.append(Finding("", "corpus", "stale",
186                                   detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                       detail=str(c.get("error") or "no reel_count
produced"))))
194             continue
195         # 2) reel count ? EXACT
196         bc, cc = b.get("reel_count"), c.get("reel_count")
197         if bc is not None and cc is not None and cc != bc:
198             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199         # 3) quality metrics ? tolerance (only if both sides have them)
200         bq, cq = b.get("quality") or {}, c.get("quality") or {}
201         for m in QUALITY_HIGHER:
202             bv, cv = bq.get(m), cq.get(m)
203             if bv is None or cv is None:
204                 continue
205             if cv < bv - tols.quality_tol:
206                 res.findings.append(Finding(v, m, "regression", bv, cv))
207             elif cv > bv + tols.quality_tol:
208                 res.findings.append(Finding(v, m, "improvement", bv, cv))
209     return res
210
211
212     # ----- #
213     # Report formatting (pure).
214     # ----- #
215     def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216         if not qd or qd.get(key) is None:
217             return "?"
218         return f"{qd[key]:.3f}"
219
220
221     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                      result: NightlyResult, date: str, head: str) -> str:
223         status = result.overall_status()
224         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                 "STALE": "? STALE (re-freeze the baseline)"}[status]
226         cost = current.get("cost", {})
227         L: List[str] = [
228             f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229             "",
230             f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
231             "",
232             f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233             f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})** · "

```

```

234         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}hr",
235         "",
236     ]
237     if result.regressions:
238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239     if result.stale:
240         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", "",
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"]
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc"?'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_{q}(bq, 'tol_f1')}"_{q}(cq, 'tol_f1')}"
255         f5 = f"_{q}(bq, 'f1@0.5')}"_{q}(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else ""
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_" \
262             + [f"- {f.message()}" for f in result.improvements] + [""]
263     L += ["---",
264         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266     return "\n".join(L)
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))

```

```

290         except (ValueError, IndexError):
291             continue
292     return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
`add_play_segment`?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()

```

```

345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
372                      if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                  rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.

```

```

'indexing.motion_threshold')."""
406     parts = dotted.split(".")
407     obj = config
408     for p in parts[:-1]:
409         obj = getattr(obj, p, None)
410         if obj is None:
411             return
412     try:
413         setattr(obj, parts[-1], value)
414     except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                       "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                       help="authoritative VM uptime (boot?delete) for the cost line; "
459                       "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                       help="freeze the current reel counts + quality as the new baseline
instead of checking")

```

```

464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
501         segmenter = manifest.get("segmenter", "motion")
502         snapshot = summarize_results(run_results, segmenter, cost)
503         snapshot["git_commit"] = _git_head()
504         snapshot["generated_at"] = date
505
506         if args.update_baseline:
507             save_baseline(args.baseline, snapshot)
508             log.info("baseline frozen ? %s", args.baseline)
509             for r in run_results:
510                 log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511             return 0
512
513         if baseline is None:
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519

```

```

520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                             "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                             fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554

```

4. user (2026-06-22T09:48:36.353Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12    """
13    from __future__ import annotations
14

```

```

15     import os
16     import tempfile
17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local", "resolve_input_ref",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                            scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull

```

```

pattern the
68     worker's ``cmd_infer`` already ships."""
69     sc = _storage_dict(storage_config)
70     ref = resolve_input_ref(raw, sc)
71     if ref.backend == "local":
72         return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74     return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->

```



```

53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57                               f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

6. assistant (2026-06-22T09:48:39.573Z)

Now let me check lines 277-278 of nightly_reelcount.py and the fetch signature more carefully:

7. user (2026-06-22T09:48:41.144Z)

```

272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
274         rallies CSV
275         (`start,end` columns / pairs). Returns None when no labels are configured."""
276         if not labels_ref:
277             return None
278         from backend.storage import fetch
279         local = fetch(labels_ref)
280         if local.endswith(".json"):
281             with open(local, encoding="utf-8") as fh:
282                 data = json.load(fh)

```

8. assistant (2026-06-22T09:48:43.066Z)

Now let me also verify the claim about line 329 where fetch is also called in run_one_video:

9. user (2026-06-22T09:48:44.110Z)

```

325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
332             {e}",
333                     "quality": None, "wall_seconds": 0.0}
334
335         gts = _load_gts(video.get("labels_uri"))

```

10. assistant (2026-06-22T09:48:46.066Z)

Let me verify the fetch() function signature again in storage/__init__.py around line 106:

11. user (2026-06-22T09:48:47.158Z)

```

104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve `uri` to a local file path. For `local` this is an identity
109         passthrough
110         (no copy); for s3/gcs it downloads to `dst`."""
111         ref = parse_uri(uri)

```

```
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
```

12. assistant (2026-06-22T09:48:49.123Z)

Now let me check the gcs.py fetch_to_local signature to verify it matches:

13. user (2026-06-22T09:48:50.176Z)

```
29
30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         if dst is None:
32             raise ValueError("GCStorage.fetch_to_local requires a dst path")
33         import os
34         dst = str(dst)
35         if os.path.exists(dst) and not overwrite:
36             return dst
37         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38         self._blob(ref).download_to_filename(dst)
39         return dst
40
```

14. assistant (2026-06-22T09:48:52.566Z)

This is critical. Let me examine the local storage backend as well to understand the full picture:

15. user (2026-06-22T09:48:53.183Z)

backend\storage\local.py

16. user (2026-06-22T09:48:55.413Z)

```
1     """Local-filestorage storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
4     (the file is already local), ``put`` is a copy (a no-op when src == dst). With
5     ``backend='local'``
6     the whole pipeline behaves exactly as before the storage layer existed.
7     """
8     from __future__ import annotations
9
10    import os
11    import shutil
12    from datetime import datetime
13    from typing import Iterator
14
15    from backend.storage.base import StorageBackend
16    from backend.storage.ref import StorageRef, StorageStat
17
18    class LocalStorage(StorageBackend):
19        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21            self.base_dir = base_dir
22
23        def _path(self, ref: StorageRef) -> str:
24            p = ref.key
25            if self.base_dir and not os.path.isabs(p):
26                return os.path.join(self.base_dir, p)
27            return p
28
29        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
```

```

30         # The object is already on the local FS ? return its path (identity). ``dst`` is
31         # honoured only if it names a *different* location (a copy is then made).
32         src = self._path(ref)
33         if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34             return src
35         dst = str(dst)
36         if os.path.exists(dst) and not overwrite:
37             return dst
38         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39         shutil.copyfile(src, dst)
40         return dst
41
42     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43         dst = self._path(ref)
44         if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46             shutil.copyfile(str(local_path), dst)
47         return ref
48
49     def exists(self, ref) -> bool:
50         return os.path.exists(self._path(ref))
51
52     def stat(self, ref) -> StorageStat:
53         st = os.stat(self._path(ref))
54         return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56     def list(self, prefix) -> Iterator[StorageRef]:
57         base = self._path(prefix)
58         if os.path.isdir(base):
59             for root, _dirs, files in os.walk(base):
60                 for fn in sorted(files):
61                     full = os.path.join(root, fn)
62                     yield StorageRef("local", "", full, f"local://{full}")
63         elif os.path.exists(base):
64             yield StorageRef("local", "", base, f"local://{base}")
65
66     def delete(self, ref) -> None:
67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

17. assistant (2026-06-22T09:48:57.554Z)

Now let me check the manifest and baseline files to understand the URIs being passed:

18. user (2026-06-22T09:48:58.859Z)

```

1      {
2      " _doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3      "segmenter": "wasb_hybrid",
4      "detector_impl": "native",
5      "config_overrides": {
6      "indexing.default_segmenter": "wasb_hybrid",
7      "indexing.detector_impl": "native"
8      },
9      "machine_usd_per_hr": 0.70,

```

```

10     "fx_inr_per_usd": 83.0,
11     "videos": [
12         {
13             "name": "GX030094",
14             "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX030094.mp4",
15             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16         },
17         {
18             "name": "Badminton_BXH_2",
19             "uri": "gs://khehsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21         },
22         {
23             "name": "GX010128",
24             "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX010128.mp4",
25             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26         }
27     ]
28 }
29

```

19. assistant (2026-06-22T09:49:11.008Z)

Perfect. Now I have all the pieces. Let me analyze the finding adversarially:

```

**The claim states:**
- `fetch(uri, dst=None, config=None, overwrite=False)` signature is correct (line 106-111 in
`__init__.py`)
- For `gs://` URIs, `fetch()` dispatches to `GCSStorage.fetch_to_local()` (lines 30-39 in
`gcs.py`)
- Calls at line 277-278 in `nightly_reelcount.py` pass only `uri` (no `dst`)
- "both handled correctly by the URI-driven backend"
- **Conclusion: "No fix needed. Storage integration is correct."**

```

Let me trace the actual execution flow:

```

1. **Line 278 in nightly_reelcount.py:** `local = fetch(labels_ref)` ? calls with only
`labels_ref` (a gs:// URI)
2. **Lines 106-111 in storage/__init__.py:** `fetch()` calls `get_storage(ref.backend,
config=config).fetch_to_local(ref, dst, overwrite=overwrite)` where `dst=None`
3. **Lines 30-32 in gcs.py:** `fetch_to_local()` **RAISES**
`ValueError("GCSStorage.fetch_to_local requires a dst path")` when `dst is None`

```

This is a **REAL BUG**. The call at line 278 will fail when `labels\_uri` is a gs:// path because GCSStorage requires dst to be specified but the code passes None.

Similarly, line 329 in `nightly\_reelcount.py` calls `fetch(video["uri"])` with no dst, and if `video["uri"]` is a gs:// path (which it is in the manifest?see line 14), this will also crash.

Let me verify the gs:// parsing to confirm the backend detection:

20. user (2026-06-22T09:49:11.251Z)

backend\storage\ref.py

21. user (2026-06-22T09:49:14.076Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A `StorageRef` is a backend-agnostic pointer to one object. URIs:
4

```

```

5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local:///<path>             -> local    (local:///abs or local://rel/p)
7         s3:///<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8         gcs:///<bucket>/<key>       -> gcs      (gs:// accepted, normalized to gcs://)
9         gdrive:///<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25     @dataclass(frozen=True)
26     class StorageStat:
27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38         backend: str
39         bucket: str
40         key: str
41         uri: str
42
43         @property
44         def name(self) -> str:
45             return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48     def parse_uri(uri: str) -> StorageRef:
49         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51         A string with no ``scheme:///`` prefix (including Windows ``C:\\...`` paths, which
52         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53         """
54         s = str(uri)
55         m = _SCHEME_RE.match(s)
56         if not m:
57             return StorageRef("local", "", s, f"local://{s}")
58         scheme = m.group(1).lower()
59         rest = s[m.end():]
60         if scheme == "gs":
61             scheme = "gcs"
62         if scheme in ("local", "gdrive"):

```

```

63         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
79         ``"gcs://bucket/x"``).
80         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
81
82     So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
83     path used today, preserving byte-for-byte behaviour.
84     """
85     s = str(raw)
86     if _SCHEME_RE.match(s):          # 1. explicit scheme wins
87         return s
88     # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
89     #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
90     #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
91     if s.startswith(("/", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
92         return s
93     if input_root:                  # 3. join a relative name under the configured
root
94         return input_root.rstrip("/") + "/" + s.lstrip("/")
95     if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
96         return f"{input_backend}:{s}"
97     return s                        # 5. bare/relative local path ? unchanged
98
99

```

22. assistant (2026-06-22T09:49:16.951Z)

Perfect. Now I have complete evidence. Let me confirm by checking if there's a local storage override or wrapper that might handle this case. Let me check the default backend in the code:

23. user (2026-06-22T09:49:18.417Z)

```

backend/storage/__init__.py:    return fetch(ref.uri, os.path.join(scratch, ref.name or
"in.mp4"), config=sc)
backend/storage/__init__.py:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,

```

24. assistant (2026-06-22T09:49:20.350Z)

Let me check if there's any config being passed that might provide a temp dir:

25. user (2026-06-22T09:49:21.425Z)

```
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                     rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
```

26. assistant (2026-06-22T09:49:24.410Z)

Now let me check what config is being passed to run_one_video and whether it's being passed to fetch():

27. user (2026-06-22T09:49:25.276Z)

```
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                         rerun_on_mismatch: bool = True,
311                         baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
```

```

338         "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340     def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341         random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342         db = Database(db_path)
343         video_id = compute_video_id(local)

```

28. assistant (2026-06-22T09:49:28.756Z)

The config is passed to `run\_one\_video` but **NOT** passed to the `fetch()` calls at lines 278 and 329. Let me verify what happens with s3:// and gdrive:// to see if this is a broader issue:

29. user (2026-06-22T09:49:29.218Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e\backend\storage'

30. user (2026-06-22T09:49:31.403Z)

```

backend\storage\__init__.py
backend\storage\base.py
backend\storage\gcs.py
backend\storage\gdrive.py
backend\storage\local.py
backend\storage\ref.py
backend\storage\s3.py

```

31. user (2026-06-22T09:49:33.741Z)

```

1     """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3     One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4     only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5     ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7     Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8     style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9     / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10    (tests pass a fake; production lets the constructor build one via boto3).
11    """
12    from __future__ import annotations
13
14    import os
15    from typing import Any, Iterator, Optional
16
17    from backend.storage.base import StorageBackend
18    from backend.storage.ref import StorageRef, StorageStat
19
20
21    class S3Storage(StorageBackend):
22        def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                    region: Optional[str] = None, addressing_style: str = "auto",
24                    signed_url_ttl: int = 3600, **_ignored) -> None:
25            self._ttl = signed_url_ttl
26            if client is not None:
27                self._client = client
28            return
29        import boto3 # type: ignore # lazy: only when an s3:// URI is actually used

```

```

30         from botocore.config import Config # type: ignore
31         self._client = boto3.client(
32             "s3",
33             endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34             region_name=region,
35             config=Config(s3={"addressing_style": addressing_style}),
36         )
37
38     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39         if dst is None:
40             raise ValueError("S3Storage.fetch_to_local requires a dst path")
41         dst = str(dst)
42         if os.path.exists(dst) and not overwrite:
43             return dst
44         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45         self._client.download_file(ref.bucket, ref.key, dst)
46         return dst
47
48     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49         extra = {"ContentType": content_type} if content_type else None
50         self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51         return ref
52
53     def exists(self, ref) -> bool:
54         try:
55             self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56             return True
57         except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58             return False
59
60     def stat(self, ref) -> StorageStat:
61         h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62         return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                             etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65     def list(self, prefix) -> Iterator[StorageRef]:
66         paginator = self._client.get_paginator("list_objects_v2")
67         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68             for obj in page.get("Contents", []) or []:
69                 key = obj["Key"]
70                 yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")
71
72     def delete(self, ref) -> None:
73         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76         op = "get_object" if method.upper() == "GET" else "put_object"
77         return self._client.generate_presigned_url(
78             op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79

```

32. user (2026-06-22T09:49:36.812Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
4     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
5     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So

```

```

6      ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7      your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8      ``local`` backend uses).
9
10     gdrive://Sharing/clip.mp4 -> <mount>/My Drive/Sharing/clip.mp4
11
12     Why an alias instead of just a raw path? Portability: the mount root differs per machine
13     (Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     ""
22     from __future__ import annotations
23
24     import glob
25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "

```

```

54         f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55     )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f" This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f" (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
69 Drive for
70 Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
71 Windows
72 ``<letter>:\My Drive`` (``G:`` is the install default). Linux/other ? configure
73 explicitly."""
74         home = os.path.expanduser("~")
75         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
76 "GoogleDrive*", "My Drive"))):
77             if os.path.isdir(p):
78                 return p
79         if os.name == "nt":
80             # G is the Drive-for-Desktop default; then scan the rest of the assignable
81 letters.
82             for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
83                 p = f"{letter}:\My Drive"
84                 if os.path.isdir(p):
85                     return p
86         return None
87
88
89     class GDriveStorage(StorageBackend):
90         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
91             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
92 _autodetect_mount_root()
93             if not root or not os.path.isdir(root):
94                 raise RuntimeError(_not_mounted_msg(root))
95             self._root = os.path.abspath(root)
96             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
97 under the mount
98
99         def _local_ref(self, ref: StorageRef) -> StorageRef:
100             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
101 relative to
102 the mount root (``LocalStorage`` joins it under ``base_dir``)."""
103             rel = ref.key.lstrip("/")
104             key = os.path.join(*rel.split("/")) if rel else ""
105             return StorageRef("local", "", key, f"local://{key}")
106
107         def _require_present(self, ref: StorageRef) -> None:
108             """Guard the read paths: a path missing from the mount almost always means the
109 folder
110 isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
111 downstream."""
112             rel = ref.key.lstrip("/")
113             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
114             if not os.path.exists(abs_path):
115                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
116
117         def _to_gdrive_uri(self, abs_path: str) -> str:
118             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""

```

```

109         rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110         return f"gdrive://{rel}"
111
112     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113         # The file is already local (via the mount): no dst => identity passthrough to
the mount
114         # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115         self._require_present(ref)
116         return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119         self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120         return ref
121
122     def exists(self, ref) -> bool:
123         return self._local.exists(self._local_ref(ref))
124
125     def stat(self, ref) -> StorageStat:
126         self._require_present(ref)
127         return self._local.stat(self._local_ref(ref))
128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

33. assistant (2026-06-22T09:49:47.519Z)

Now I have complete clarity. Let me summarize the defect:

****EVIDENCE OF THE BUG:****

- **GCS backend (lines 30-32 in gcs.py):****

```

```python
def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
 if dst is None:
 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
...

```
- \*\*S3 backend (lines 38-40 in s3.py):\*\***  

```

```python
def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
    if dst is None:
        raise ValueError("S3Storage.fetch_to_local requires a dst path")
...

```
- **Calls in nightly_reelcount.py:****
 - Line 278: ``local = fetch(labels_ref)`` ? no dst provided
 - Line 329: ``local = fetch(video["uri"])`` ? no dst provided
- **Manifest uses gs:// URIs (lines 14-15 in reelcount_manifest.json):****

```

```json
"uri": "gs://kheldutra-rally-corpus/nightly/videos/GX030094.mp4",
"labels_uri": "gs://kheldutra-rally-corpus/nightly/labels/GX030094_golden_features.json"
...

```
- \*\*URI parsing (lines 60-61 in ref.py):\*\***  

```

```python
if scheme == "gs":

```

```
... scheme = "gcs"
```

So `gs://` URIs resolve to the `gcs` backend, which requires `dst`.

****THE BUG:**** When the nightly runner executes with `gs://` URIs (as configured in the manifest), both `fetch()` calls will crash with `ValueError("GCSStorage.fetch\_to\_local requires a dst path")`.

The ****fix**** is to provide a temporary destination path to the fetch calls when dealing with remote URIs, or use the `acquire\_local\_input()` helper which already handles this (line 74 in `\_\_init\_\_.py`).

Now let me call StructuredOutput with the findings:

34. user (2026-06-22T09:49:58.485Z)

Structured output provided successfully

35. assistant (2026-06-22T09:50:09.880Z)

Summary

The finding you asked me to verify is ****INCORRECT**** ? I found a ****REAL, HIGH-SEVERITY BUG**** that contradicts it.

****The Bug:**** The nightly reel-count engine will ****crash at runtime**** when processing `gs://` URIs (which are configured in the manifest), because both `fetch()` calls at lines 278 and 329 omit the required `dst` parameter. GCSStorage and S3Storage both raise `ValueError` when `dst` is `None`.

****Evidence:****

- GCS backend requires `dst` (gcs.py:30-32)
- S3 backend requires `dst` (s3.py:38-40)
- Manifest uses ONLY `gs://` URIs (reelcount_manifest.json:14-15, 19-20, 24-25)
- Calls provide no `dst` (nightly_reelcount.py:278, 329)

****Fix:**** Use `acquire\_local\_input()` helper (already exists in storage/__init__.py:59-74) which properly allocates temp directories for remote URIs while identity-passing local paths.

This is a correctness bug that will cause complete pipeline failure on the first nightly run?not a style issue.